

Tutorial Genérico: Comunicação entre Microserviços com Apache Kafka

1. Definir o fluxo

Antes de criar os arquivos, defina claramente:

- Qual serviço inicia o processo;
- Qual evento será publicado;
- Qual serviço consumirá esse evento;
- Qual resultado será devolvido;
- Quais estados a entidade poderá assumir.

Exemplo genérico:

```
Serviço A
↓
Cria uma entidade com status PENDING
↓
Publica EventRequested
↓
Serviço B consome
↓
Executa a operação
↓
Publica EventResult
↓
Serviço A consome
↓
Atualiza o status
```

Exemplo prático:

```
Order Service
↓
Publica solicitação de reserva
↓
Inventory Service
↓
Reserva o recurso
↓
Publica sucesso ou falha
↓
Order Service
```

↓
Atualiza o pedido

2. Adicionar a dependência do Kafka

Nos dois microsserviços, adicione ao `pom.xml` :

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

3. Criar a estrutura de diretórios

Em cada microsserviço, crie uma pasta de mensageria:

```
src/main/java/com/exemplo/servico
├── config
│   └── KafkaConfig.java
├── messaging
│   ├── consumer
│   ├── event
│   ├── mapper
│   └── publisher
└── services
```

A estrutura pode variar conforme a responsabilidade do serviço.

Serviço produtor e consumidor

```
messaging
├── consumer
│   └── ResultEventConsumer.java
├── event
│   ├── RequestEvent.java
│   ├── RequestItemEvent.java
│   └── ResultEvent.java
├── mapper
│   └── EventMapper.java
└── publisher
```

```
└─ EventPublisher.java
└─ EventDispatcher.java
```

Serviço apenas consumidor e produtor de resposta

```
messaging
└─ consumer
    └─ RequestEventConsumer.java
└─ event
    └─ RequestEvent.java
    └─ RequestItemEvent.java
    └─ ResultEvent.java
└─ publisher
    └─ ResultEventPublisher.java
```

4. Criar os contratos dos eventos

Os eventos devem representar contratos de comunicação entre os serviços.

Evite enviar diretamente:

- Entidades JPA;
- Objetos de domínio completos;
- DTOs específicos de controller;
- Objetos com dependências internas do serviço.

Evento de item

```
package com.exemplo.servico.messaging.event;

public record RequestItemEvent(
    String resourceId,
    String resourceCode
) {
}
```

Evento de solicitação

```
package com.exemplo.servico.messaging.event;

import java.time.LocalDateTime;
import java.util.List;

public record RequestEvent(
```

```

        String messageId,
        String processId,
        String userId,
        List<RequestItemEvent> items,
        LocalDateTime occurredAt
    ) {
    }

```

Evento de resultado

```

package com.exemplo.servico.messaging.event;

import java.time.LocalDateTime;
import java.util.List;

public record ResultEvent(
    String messageId,
    String processId,
    boolean success,
    List<String> processedResources,
    String reason,
    LocalDateTime occurredAt
) {
}

```

Os dois microsserviços devem possuir contratos compatíveis, com os mesmos:

- Nomes de propriedades;
- Tipos;
- Estrutura;
- Significado.

Os pacotes Java podem ser diferentes.

5. Criar o mapper no serviço produtor

O mapper transforma a entidade interna no evento Kafka.

```

package com.exemplo.servico.messaging.mapper;

import com.exemplo.servico.entities.ProcessEntity;
import com.exemplo.servico.messaging.event.RequestEvent;
import com.exemplo.servico.messaging.event.RequestItemEvent;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;

```

```

import java.util.List;
import java.util.UUID;

@Component
public class EventMapper {

    public RequestEvent toRequestEvent(ProcessEntity process) {
        List<RequestItemEvent> items = process.getItems()
            .stream()
            .map(item -> new RequestItemEvent(
                item.getResourceId(),
                item.getResourceCode()
            ))
            .toList();

        return new RequestEvent(
            UUID.randomUUID().toString(),
            process.getId(),
            process.getUserId(),
            items,
            LocalDateTime.now()
        );
    }
}

```

O objetivo é desacoplar o modelo interno do contrato de mensageria.

6. Configurar o producer e o consumer

Cada serviço pode publicar e consumir eventos.

Configuração genérica

```

package com.exemplo.servico.config;

import com.exemplo.servico.messaging.event.ResultEvent;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.apache.kafka.common.serialization.StringSerializer;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.ConcurrentKafkaListenerContainerFactory;
import org.springframework.kafka.core.*;
import org.springframework.kafka.support.serializer.JsonDeserializer;

```

```

import org.springframework.kafka.support.serializer.JsonSerializer;

import java.util.HashMap;
import java.util.Map;

@Configuration
public class KafkaConfig {

    @Value("${app.kafka.server-url}")
    private String kafkaServerUrl;

    @Value("${app.kafka.consumer-group}")
    private String consumerGroup;

    @Bean
    public ProducerFactory<String, Object> producerFactory() {
        Map<String, Object> properties = new HashMap<>();

        properties.put(
            ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
            kafkaServerUrl
        );

        properties.put(
            ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
            StringSerializer.class
        );

        properties.put(
            ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
            JsonSerializer.class
        );

        properties.put(
            JsonSerializer.ADD_TYPE_INFO_HEADERS,
            false
        );

        return new DefaultKafkaProducerFactory<>(properties);
    }

    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate(
        ProducerFactory<String, Object> producerFactory
    ) {
        return new KafkaTemplate<>(producerFactory);
    }

    @Bean
    public ConsumerFactory<String, ResultEvent> resultConsumerFactory() {
        Map<String, Object> properties = new HashMap<>();

```

```

        properties.put(
            ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
            kafkaServerUrl
        );

        properties.put(
            ConsumerConfig.GROUP_ID_CONFIG,
            consumerGroup
        );

        properties.put(
            ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
            "earliest"
        );

        properties.put(
            ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            StringDeserializer.class
        );

        JsonSerializer<ResultEvent> deserializer =
            new JsonSerializer<>(ResultEvent.class, false);

        deserializer.addTrustedPackages(
            "com.exemplo.servico.messaging.event"
        );

        return new DefaultKafkaConsumerFactory<>(
            properties,
            new StringDeserializer(),
            deserializer
        );
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, ResultEvent>
    resultKafkaListenerContainerFactory() {
        ConcurrentKafkaListenerContainerFactory<String, ResultEvent> factory
        =
            new ConcurrentKafkaListenerContainerFactory<>();

        factory.setConsumerFactory(resultConsumerFactory());

        return factory;
    }
}

```

No outro microserviço, altere o tipo consumido:

```
ConsumerFactory<String, RequestEvent>
```

7. Configurar o YAML

Exemplo genérico:

```
app:
  kafka:
    server-url: localhost:29092
    consumer-group: service-a-group
    topics:
      request: application.process.requested.v1
      result: application.process.result.v1
```

Para outro serviço:

```
app:
  kafka:
    server-url: localhost:29092
    consumer-group: service-b-group
    topics:
      request: application.process.requested.v1
      result: application.process.result.v1
```

Os nomes dos tópicos devem ser iguais nos dois serviços.

Os grupos devem ser diferentes:

```
service-a-group
service-b-group
```

Endereço do Kafka

Aplicação executada pela IDE:

```
server-url: localhost:29092
```

Aplicação executada dentro do Docker:

```
server-url: kafka:9092
```


Não utilize:

```
server-url: http://localhost:29092
```

Kafka não utiliza HTTP nesse endereço.

8. Criar o publisher

```
package com.exemplo.servico.messaging.publisher;

import com.exemplo.servico.messaging.event.RequestEvent;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

@Component
public class EventPublisher {

    private static final Logger logger =
        LoggerFactory.getLogger(EventPublisher.class);

    private final KafkaTemplate<String, Object> kafkaTemplate;
    private final String requestTopic;

    public EventPublisher(
        KafkaTemplate<String, Object> kafkaTemplate,
        @Value("${app.kafka.topics.request}")
        String requestTopic
    ) {
        this.kafkaTemplate = kafkaTemplate;
        this.requestTopic = requestTopic;
    }

    public void publish(RequestEvent event) {
        kafkaTemplate.send(
            requestTopic,
            event.processId(),
            event
        ).whenComplete((result, exception) -> {
            if (exception != null) {
                logger.error(
                    "Erro ao publicar evento do processo {}.\"",
                    event.processId(),
                    exception
                );
            }
        });
    }
}
```

```

        return;
    }

    logger.info(
        "Evento do processo {} publicado. Partição: {}. Offset: {}.",
        event.processId(),
        result.getRecordMetadata().partition(),
        result.getRecordMetadata().offset()
    );
});
}
}

```

A chave da mensagem deve representar o agregado ou processo principal:

```
event.processId()
```

Isso ajuda a manter eventos relacionados na mesma partição.

9. Publicar após a transação

Quando o serviço salva uma entidade no banco e depois publica um evento, evite publicar antes do commit.

Crie um dispatcher:

```

package com.exemplo.servico.messaging.publisher;

import com.exemplo.servico.messaging.event.RequestEvent;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

@Component
public class EventDispatcher {

    private final EventPublisher eventPublisher;

    public EventDispatcher(EventPublisher eventPublisher) {
        this.eventPublisher = eventPublisher;
    }

    @TransactionalEventListener(
        phase = TransactionPhase.AFTER_COMMIT
    )
    public void dispatch(RequestEvent event) {

```

```

        eventPublisher.publish(event);
    }
}

```

No service, use o `ApplicationEventPublisher`:

```
private final ApplicationEventPublisher applicationEventPublisher;
```

Depois de salvar:

```

RequestEvent event = eventMapper.toRequestEvent(savedEntity);

applicationEventPublisher.publishEvent(event);

```

Fluxo:

```

salva entidade
  ↓
commit no banco
  ↓
dispatcher recebe
  ↓
publisher envia ao Kafka

```

10. Criar o consumer no segundo serviço

```

package com.exemplo.outroservico.messaging.consumer;

import com.exemplo.outroservico.messaging.event.RequestEvent;
import com.exemplo.outroservico.messaging.publisher.ResultEventPublisher;
import com.exemplo.outroservico.services.ResourceService;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
public class RequestEventConsumer {

    private final ResourceService resourceService;
    private final ResultEventPublisher resultEventPublisher;

    public RequestEventConsumer(
        ResourceService resourceService,

```

```

        ResultEventPublisher resultEventPublisher
    ) {
        this.resourceService = resourceService;
        this.resultEventPublisher = resultEventPublisher;
    }

    @KafkaListener(
        topics = "${app.kafka.topics.request}",
        groupId = "${app.kafka.consumer-group}",
        containerFactory = "requestKafkaListenerContainerFactory"
    )
    public void consume(RequestEvent event) {
        List<String> resourceCodes = event.items()
            .stream()
            .map(item -> item.resourceCode())
            .toList();

        try {
            List<String> processedResources =
                resourceService.processResources(
                    event.userId(),
                    resourceCodes
                );

            resultEventPublisher.publishSuccess(
                event.processId(),
                processedResources
            );
        } catch (RuntimeException exception) {
            String reason = exception.getMessage();

            if (reason == null || reason.isBlank()) {
                reason = "Não foi possível processar os recursos.";
            }

            resultEventPublisher.publishFailure(
                event.processId(),
                resourceCodes,
                reason
            );
        }
    }
}

```

11. Criar uma operação coordenadora no serviço consumidor

Quando um evento possuir vários itens, crie um método que coordene o processamento.

```
public List<String> processResources(
    String userId,
    List<String> resourceCodes
) {
    if (resourceCodes == null || resourceCodes.isEmpty()) {
        throw new BusinessException(
            "Nenhum recurso foi informado."
        );
    }

    List<String> processedResources = new ArrayList<>();

    try {
        for (String resourceCode : resourceCodes) {
            processResource(resourceCode, userId);
            processedResources.add(resourceCode);
        }

        return List.copyOf(processedResources);
    } catch (RuntimeException exception) {
        for (String processedResource : processedResources) {
            try {
                rollbackResource(processedResource);
            } catch (RuntimeException rollbackException) {
                logger.error(
                    "Falha ao reverter o recurso {}. ",
                    processedResource,
                    rollbackException
                );
            }
        }

        throw exception;
    }
}
```

Isso evita processamento parcial.

Exemplo:

Recurso A → processado
Recurso B → processado
Recurso C → falhou

Resultado:

Reverte A
Reverte B
Publica falha

12. Criar o publisher de resultado

```
package com.exemplo.outroservico.messaging.publisher;

import com.exemplo.outroservico.messaging.event.ResultEvent;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.util.List;
import java.util.UUID;

@Component
public class ResultEventPublisher {

    private final KafkaTemplate<String, Object> kafkaTemplate;
    private final String resultTopic;

    public ResultEventPublisher(
        KafkaTemplate<String, Object> kafkaTemplate,
        @Value("${app.kafka.topics.result}")
        String resultTopic
    ) {
        this.kafkaTemplate = kafkaTemplate;
        this.resultTopic = resultTopic;
    }

    public void publishSuccess(
        String processId,
        List<String> resources
    ) {
        ResultEvent event = new ResultEvent(
            UUID.randomUUID().toString(),
            processId,
            true,

```

```

        List.copyOf(resources),
        null,
        LocalDateTime.now()
    );

    publish(event);
}

public void publishFailure(
    String processId,
    List<String> resources,
    String reason
) {
    ResultEvent event = new ResultEvent(
        UUID.randomUUID().toString(),
        processId,
        false,
        List.copyOf(resources),
        reason,
        LocalDateTime.now()
    );

    publish(event);
}

private void publish(ResultEvent event) {
    kafkaTemplate.send(
        resultTopic,
        event.processId(),
        event
    );
}
}

```

13. Criar o consumer de resultado

No primeiro serviço:

```

package com.exemplo.servico.messaging.consumer;

import com.exemplo.servico.messaging.event.ResultEvent;
import com.exemplo.servico.services.ProcessService;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class ResultEventConsumer {

```

```

private final ProcessService processService;

public ResultEventConsumer(ProcessService processService) {
    this.processService = processService;
}

@KafkaListener(
    topics = "${app.kafka.topics.result}",
    groupId = "${app.kafka.consumer-group}",
    containerFactory = "resultKafkaListenerContainerFactory"
)
public void consume(ResultEvent event) {
    processService.handleResult(event);
}
}

```

14. Atualizar o estado da entidade

No serviço principal:

```

@Transactional
public void handleResult(ResultEvent event) {
    ProcessEntity process = repository.findById(event.processId())
        .orElseThrow(
            () -> new NotFoundException(
                "Processo não encontrado."
            )
        );

    if (process.getStatus() != ProcessStatus.PENDING) {
        return;
    }

    if (event.success()) {
        process.updateStatus(ProcessStatus.COMPLETED);
    } else {
        process.updateStatus(ProcessStatus.FAILED);
    }
}
}

```

Exemplo de enum:

```

public enum ProcessStatus {
    PENDING,
    COMPLETED,
}

```



```
    FAILED,  
    CANCELLED  
}
```

Ao adicionar novos valores ao enum, verifique se o banco possui uma constraint `CHECK`.

15. Ajustar o banco de dados

Se houver uma constraint de status, ela precisa aceitar todos os valores do enum.

Consultar a constraint:

```
SELECT pg_get_constraintdef(oid)  
FROM pg_constraint  
WHERE conname = 'nome_da_constraint';
```

Remover:

```
ALTER TABLE nome_da_tabela  
DROP CONSTRAINT nome_da_constraint;
```

Recriar:

```
ALTER TABLE nome_da_tabela  
ADD CONSTRAINT nome_da_constraint  
CHECK (  
    status IN (  
        'PENDING',  
        'COMPLETED',  
        'FAILED',  
        'CANCELLED'  
    )  
);
```

O Hibernate pode não atualizar automaticamente constraints já existentes.

16. Ajustar o controller

Como o processamento é assíncrono, o endpoint inicial não devolve o resultado final.

Exemplo:

```

@PostMapping
public ResponseEntity<ProcessResponseDTO> create(
    @RequestBody ProcessRequestDTO request
) {
    ProcessResponseDTO response =
        processService.create(request);

    return ResponseEntity
        .accepted()
        .body(response);
}

```

O código HTTP mais adequado é:

202 Accepted

A resposta inicial pode ser:

```

{
  "id": "process-id",
  "status": "PENDING"
}

```

Depois, o cliente consulta:

`GET /processes/{id}`

Resultado posterior:

```

{
  "id": "process-id",
  "status": "COMPLETED"
}

```

ou:

```

{
  "id": "process-id",
  "status": "FAILED"
}

```

17. Ordem recomendada de implementação

Etapa 1 — Infraestrutura

1. Subir Kafka;
2. Verificar broker;
3. Verificar Kafka UI;
4. Definir nomes dos tópicos;
5. Configurar endereço e grupos.

Etapa 2 — Contratos

1. Criar evento de item;
2. Criar evento de solicitação;
3. Criar evento de resultado;
4. Garantir que os contratos sejam compatíveis nos dois serviços.

Etapa 3 — Serviço produtor

1. Criar mapper;
2. Criar publisher;
3. Criar dispatcher após commit;
4. Alterar o service para publicar o evento;
5. Remover a chamada síncrona antiga.

Etapa 4 — Serviço consumidor

1. Criar consumer;
2. Criar método coordenador;
3. Implementar compensação;
4. Criar publisher de resultado.

Etapa 5 — Retorno ao serviço inicial

1. Criar consumer de resultado;
2. Atualizar o estado da entidade;
3. Ajustar enum;
4. Ajustar constraint do banco.

Etapa 6 — Testes

1. Testar cenário de sucesso;
2. Testar recurso inexistente;
3. Testar recurso indisponível;
4. Testar falha parcial;
5. Testar atualização do status;
6. Verificar mensagens e offsets no Kafka UI.

18. Checklist final

- [] Dependência spring-kafka adicionada
- [] KafkaConfig criado nos dois serviços
- [] Producer usando JsonSerializer
- [] Consumer usando JsonDeserializer
- [] Tópicos iguais nos dois serviços
- [] Consumer groups diferentes
- [] Contratos dos eventos compatíveis
- [] Publisher criado
- [] Consumer criado
- [] Entidade não é enviada diretamente
- [] Chamada síncrona antiga foi removida
- [] Status inicial é PENDING
- [] Resultado atualiza a entidade
- [] Banco aceita os novos status
- [] Endpoint retorna 202 Accepted
- [] Cliente consulta o resultado posteriormente
- [] Falha parcial possui compensação
- [] Logs mostram tópico, partição e offset

Fluxo final genérico

```
Cliente
  ↓
POST no Serviço A
  ↓
Serviço A salva entidade como PENDING
  ↓
Serviço A publica RequestEvent
  ↓
Kafka
  ↓
Serviço B consome RequestEvent
  ↓
Serviço B executa a operação
  ↓
Serviço B publica ResultEvent
  ↓
Kafka
  ↓
Serviço A consome ResultEvent
  ↓
Serviço A atualiza para COMPLETED ou FAILED
```

↓
Cliente consulta o estado atualizado